C. COMPUTAÇÃO
UNIPAC

CENTRO UNIVERSITÁRIO UNIPAC
SCIENTIA LIBERTAS ET FIDES
PRESIDENTE ANTÔNIO CARLOS

unipac.br
Barbacena

Computação Gráfica

Pygame - Pipeline Gráfico

Professor:

Dr. Rodrigo Fenandes dos Santos

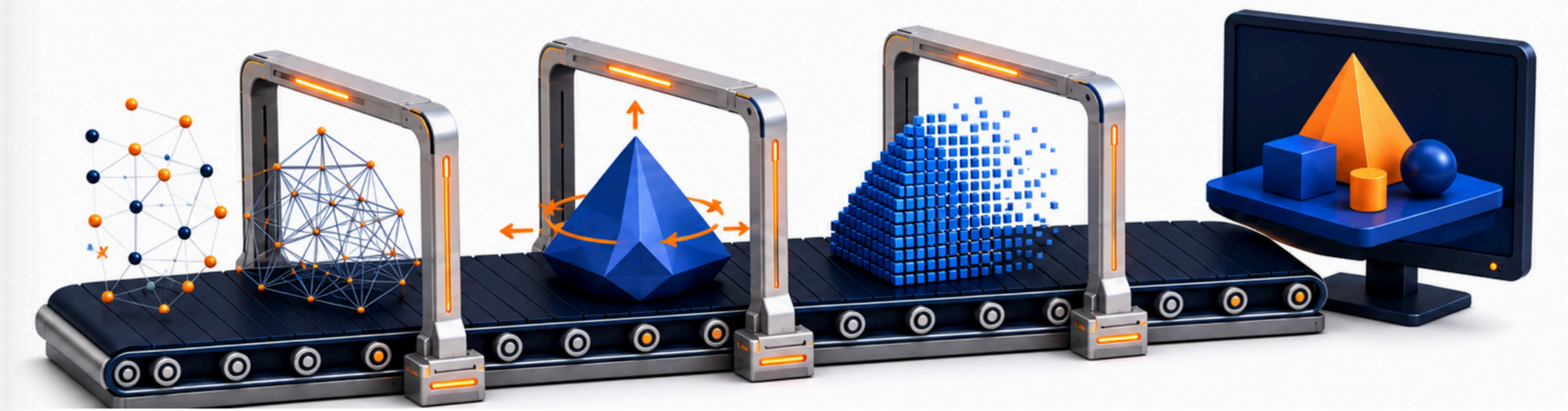
Aulas 07 e 08

unipac.br
Barbacena

C. COMPUTAÇÃO
UNIPAC

Pipeline gráfico: uma linha de produção visual

Cada etapa transforma dados da cena em uma imagem que vemos na tela.



1 APLICAÇÃO

A CPU organiza vértices, cores e parâmetros da cena.

2 GEOMETRIA

O vertex shader transforma e posiciona os vértices.

3 RASTERIZAÇÃO

Primitivas viram fragments: candidatos a pixels.

4 PROCESSAMENTO DE PIXELS

O fragment shader define a cor gravada no framebuffer.

dados da cena → vértices transformados → fragments → pixels no framebuffer

unipac.br
Barbacena

Por que o pipeline gráfico é eficiente?

Dividir o trabalho mantém vários estágios produtivos ao mesmo tempo.

01 **SEQUÊNCIA**

Um modelo abstrato organiza as etapas necessárias para renderizar uma cena tridimensional.

02 **SUBTAREFAS EM PARALELO**

Cada etapa executa parte do trabalho — como em uma linha de montagem — aumentando a eficiência do processo.

03 **MAIOR VAZÃO DE IMAGENS**

Enquanto um estágio trabalha, os demais podem avançar em outras partes do fluxo, permitindo taxas de exibição mais rápidas.

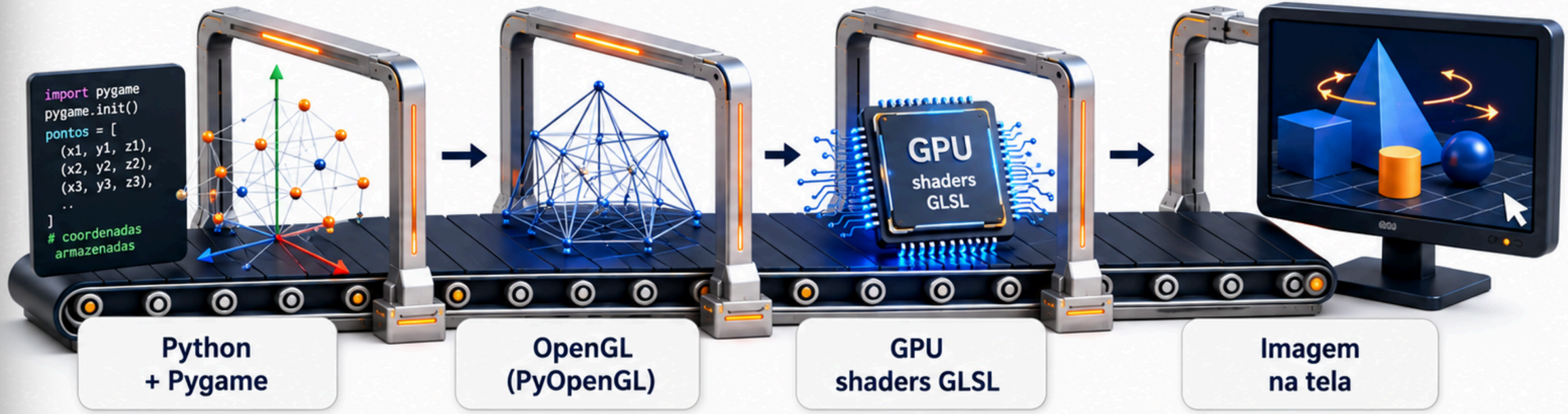
Há vários modelos de pipeline. Nesta aula, estudaremos o modelo de quatro estágios usado no OpenGL para gráficos em tempo real.

unipac.br
Barbacena

Introdução ao Pygame e OpenGL

PERGUNTA-GUIA DA AULA

“Como um conjunto de coordenadas armazenado em um programa Python se transforma em uma imagem animada e interativa exibida na tela?”



Fonte: L. Stemkoski & M. Pascale, Developing Graphics Frameworks with Python and OpenGL — Capítulo 2 (p. 25–82)

O que construiremos hoje

A aula é uma escada: cada etapa só existe porque a anterior deixou um problema em aberto.

Nada é "mágica da biblioteca": cada linha nova resolve uma limitação concreta.

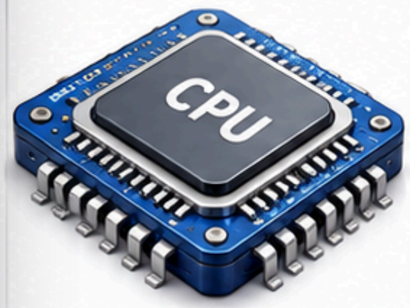
Ao final, o triângulo interativo usa todos os seis blocos ao mesmo tempo.

FUNDAMENTOS

O problema computacional

Desenhar exige cooperação entre três atores com papéis distintos.

APLICAÇÃO (CPU)



coordenadas

cores

- Guarda os dados: coordenadas e cores
- Decide o que desenhar e quando
- Controla o tempo e lê a entrada do usuário

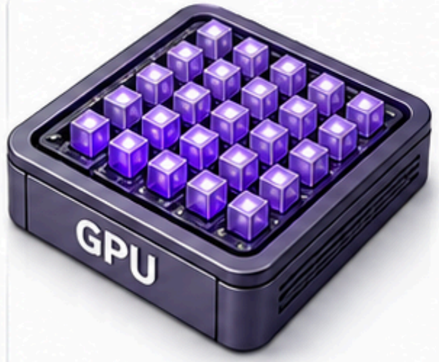
dados + comandos

API GRÁFICA

- Vocabulário comum entre aplicação e hardware
- OpenGL é a especificação; PyOpenGL é a ponte em Python
- Envia dados e comandos para a GPU

PCI Express

GPU



shaders

framebuffer

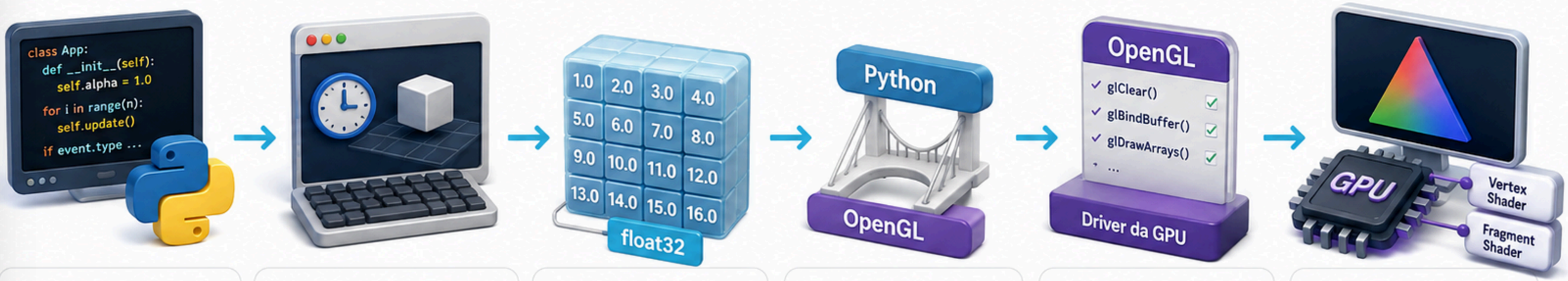
- Milhares de núcleos em paralelo
- Executa shaders por vértice e por pixel
- Escreve o resultado no framebuffer

CPU e GPU têm memórias separadas. Copiar dados para a GPU custa caro; por isso, fazemos essa transferência poucas vezes.

FUNDAMENTOS

AS TECNOLOGIAS ENVOLVIDAS

Seis nomes, seis papéis — confundi-los é a principal fonte de erro nesta unidade.



Python

Linguagem da aplicação: classes, laços, estrutura do programa.

Pygame

Cria a janela, mede o tempo e entrega eventos do teclado. Não desenha 3D.

NumPy

Converte listas Python em arrays contíguos de float de 32 bits.

PyOpenGL

Binding: expõe as funções gl* do OpenGL dentro do Python.

OpenGL

Especificação da API gráfica implementada pelo driver da GPU.

GLSL

Linguagem estilo C dos shaders, compilada e executada pela GPU.

Regra prática: Pygame cuida da janela e do usuário; OpenGL cuida do que aparece dentro dela.

unipac.br
Barbacena

C. COMPUTAÇÃO
UNIPAC

JANELA E CICLO DE VIDA

Ciclo de vida da aplicação

Toda aplicação gráfica interativa executa um laço que repete três subetapas cerca de **60 vezes por segundo**.

STARTUP
cria objetos, carrega recursos e compila shaders.

PROCESS INPUT
lê o que o usuário fez.

UPDATE
altera posições, cores e contadores; não desenha pixels.

RENDER
envia os comandos de desenho e troca os buffers.

SHUTDOWN
encerra o Pygame e o processo.

Update prepara a cena. Render transforma a cena em imagem.

PIPELINE GRÁFICO

Criação da Janela:

- O estágio da aplicativo envolve principalmente processos que são executados na CPU.
- Uma das primeiras tarefas é criar uma janela onde os gráficos renderizados serão exibidos.
- Ao trabalhar com OpenGL, isso pode ser feito usando uma variedade de linguagens de programação.
- A janela (ou um objeto semelhante a uma tela dentro da janela) deve ser inicializada para que os gráficos sejam lidos do framebuffer da GPU.

unipac.br
Barbacena

C. COMPUTAÇÃO UNIPAC

JANELA E CICLO DE VIDA

Arquitetura orientada a objetos

O capítulo não escreve scripts soltos: constrói um framework reutilizável desde a primeira linha.

ORGANIZAÇÃO DE PASTAS

pasta_principal/

core/ *(pacote)*

- __init__.py (vazio)
- base.py — Base
- input.py — Input
- openGLUtils.py — OpenGLUtils
- attribute.py — Attribute
- uniform.py — Uniform

- test-2-1.py — aplicação
- test-2-2.py … test-2-11.py

POR QUE ASSIM?

- __init__.py (vazio) faz o Python reconhecer **core** como pacote importável.
- As aplicações ficam na pasta principal e estendem **Base**: os imports do livro pressupõem essa posição.
- Cada classe encapsula um assunto: janela, entrada, compilação, atributos, uniforms.
- Organização, reusabilidade e extensibilidade — o framework cresce até o fim do livro.

O core é a fundação; cada aplicação reutiliza e amplia essa base.

unipac.br
Barbacena

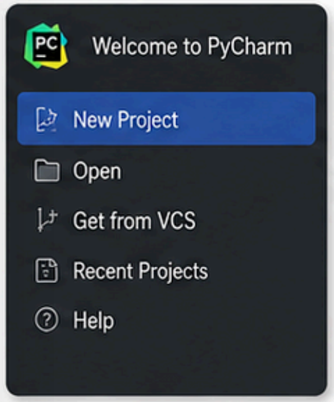
C. COMPUTAÇÃO UNIPAC

Criando o projeto e a pasta core

Organização inicial para trabalhar com Pygame e OpenGL

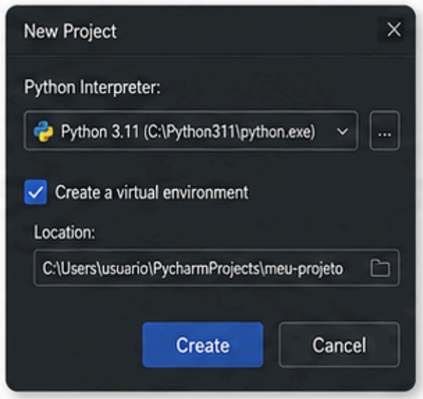
PyCharm Python Pygame core

1. Crie um novo projeto



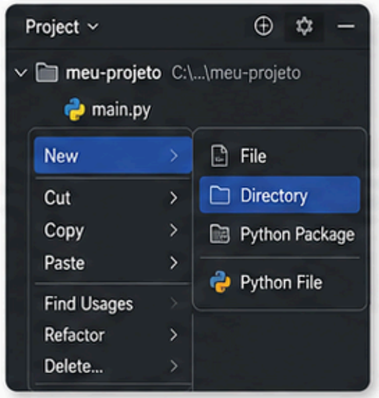
No PyCharm, clique em **New Project**.

2. Configure o Python



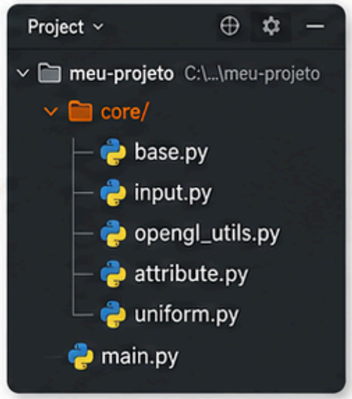
Selecione um interpretador Python e clique em Create.

3. Crie a pasta core



No projeto, clique com o botão direito → **New** → Directory → core.

4. Organize os arquivos-base



A pasta core guarda classes reutilizáveis do projeto.

Estrutura final do projeto

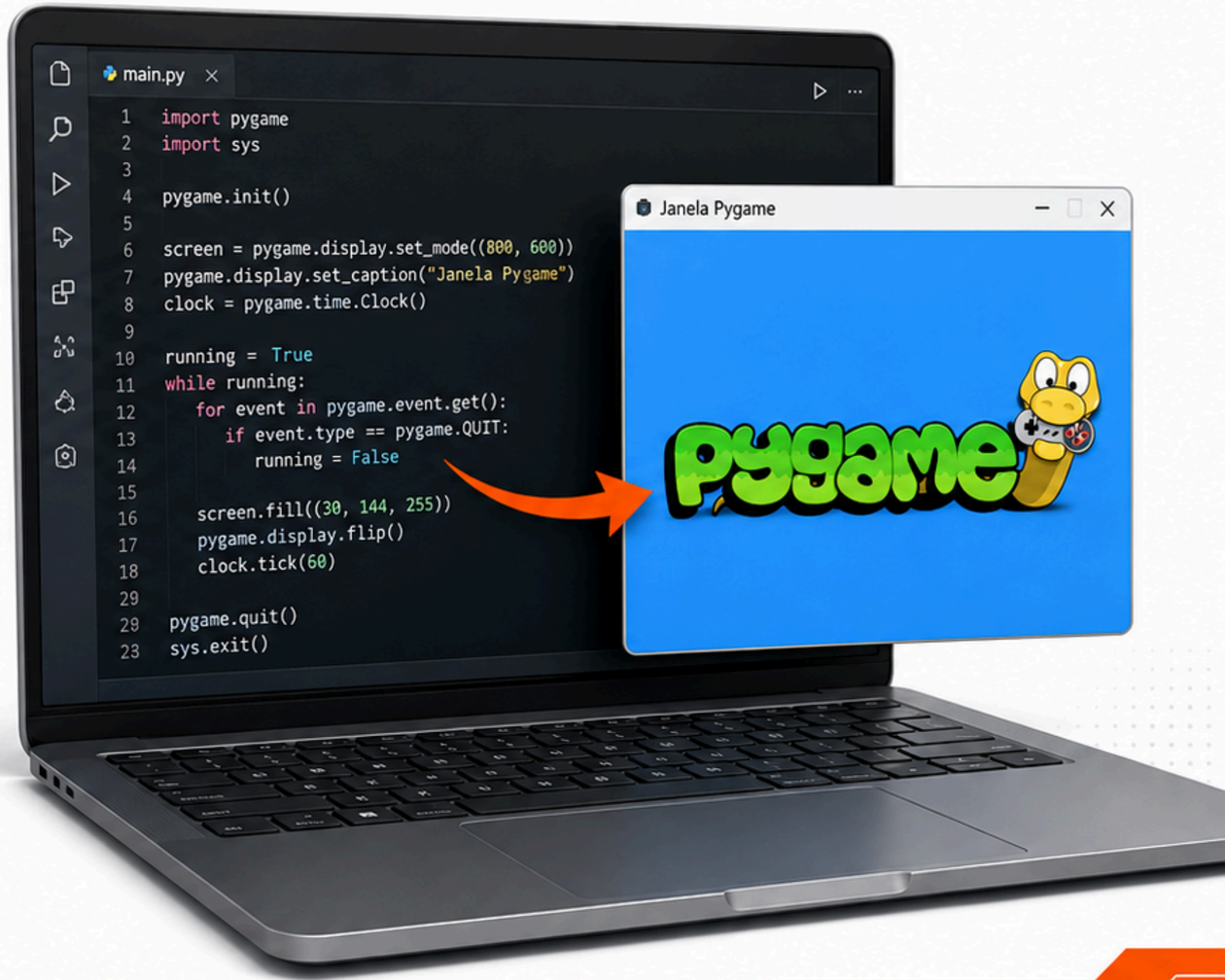
- meu-projeto/
  - core/
    - base.py
    - input.py
    - opengl_utils.py
    - attribute.py
    - uniform.py
  - main.py

IDEIA-CHAVE: a pasta core concentra o código reutilizável; main.py fica responsável por iniciar a aplicação.

CRIANDO JANELAS COM PYGAME

- Conforme indicado no pipeline de gráficos, o primeiro passo na renderização de gráficos é desenvolver um aplicativo onde os gráficos serão exibidos.

- Nós iremos criar aplicativos em janela usando Python e Pygame, uma biblioteca popular de desenvolvimento de jogos Python.



CRIANDO JANELAS COM PYGAME

- O processo de criação de uma janela em Pygame envolve a criação de uma classe chamada **Base**, que é responsável por inicializar o Pygame e exibir a janela.
- Esta classe é projetada para gerenciar o ciclo de vida padrão de uma aplicação que possua interação do usuário e animação.

unipac.br
Barbacena

C. COMPUTAÇÃO UNIPAC

Entendendo a classe Base: passo a passo

1. Importações

```
import pygame
import sys
```

Traz as bibliotecas para criar a janela, controlar o tempo e encerrar o programa.

2. Classe Base e __init__

class Base(object):
Cria um modelo reutilizável para todos os exemplos gráficos.

pygame.init()
Inicializa todos os módulos do Pygame.

DOUBLEBUF | OPENGL
Ativa janela OpenGL com buffer duplo: um quadro é preparado enquanto o outro é exibido.

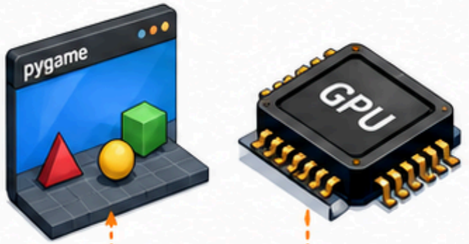
Antialiasing: 1 buffer + 4 amostras
Suaviza as bordas das formas desenhadas.

Perfil OpenGL CORE
Usa uma configuração moderna e compatível entre plataformas.

pygame.display.set_mode(screenSize, displayFlags)
Cria a janela gráfica com o tamanho definido.

set_caption("Graphics Window")
Define o título da janela.

running = True • Clock()
Controla o laço principal, FPS e delta time.




```
class Base(object):
  def __init__(self):
    # prepara janela e estado
  def initialize(self):
    # preparado pela aplicação
  def update(self, dt):
    # lógica da cena
  def run(self):
    # ciclo da aplicação
```

3. Métodos que a aplicação completa

initialize()
Onde preparamos recursos: shaders, objetos e dados.

update()
Onde alteramos o estado da cena a cada quadro.

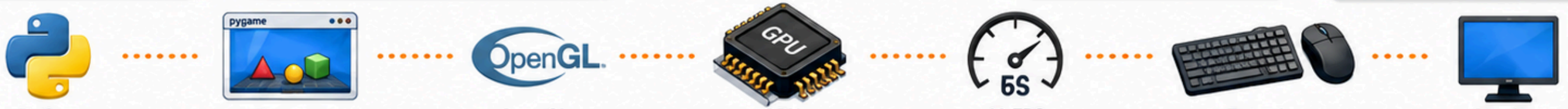
4. run(): ciclo da aplicação

initialize() →while running → input → update() → flip() → tick(60)

flip() exibe na tela o quadro que foi renderizado.

tick(60) limita o laço a até 60 quadros por segundo.

pygame.quit() + sys.exit()
Fecha o Pygame e encerra o programa com segurança.



IDEIA-CHAVE: Base prepara a janela e o ciclo; cada exemplo adiciona sua própria cena gráfica.

input.py — Capturando o fechamento da janela

O arquivo Input observa os eventos do Pygame e informa à Base quando o usuário deseja encerrar a aplicação.

1. Crie core/input.py

```python
import pygame

class Input(object):
    def __init__(self):
        # Indica se o usuário fechou a aplicação
        self.quit = False

    def update(self):
        # Lê teclado, mouse e janela desde a última verificação
        for event in pygame.event.get():
            # Clique no X da janela
            if event.type == pygame.QUIT:
                self.quit = True
```

2. Conecte à classe Base

```python
from core.input import Input
self.input = Input()
```

Importe a classe e crie uma instância no fim de __init__.

3. Verifique a entrada no loop

```python
self.input.update()
if self.input.quit:
    self.running = False
```

A cada quadro, Base atualiza Input. Se quit for True, o loop é encerrado.

Usuário clica no X

pygame.QUIT

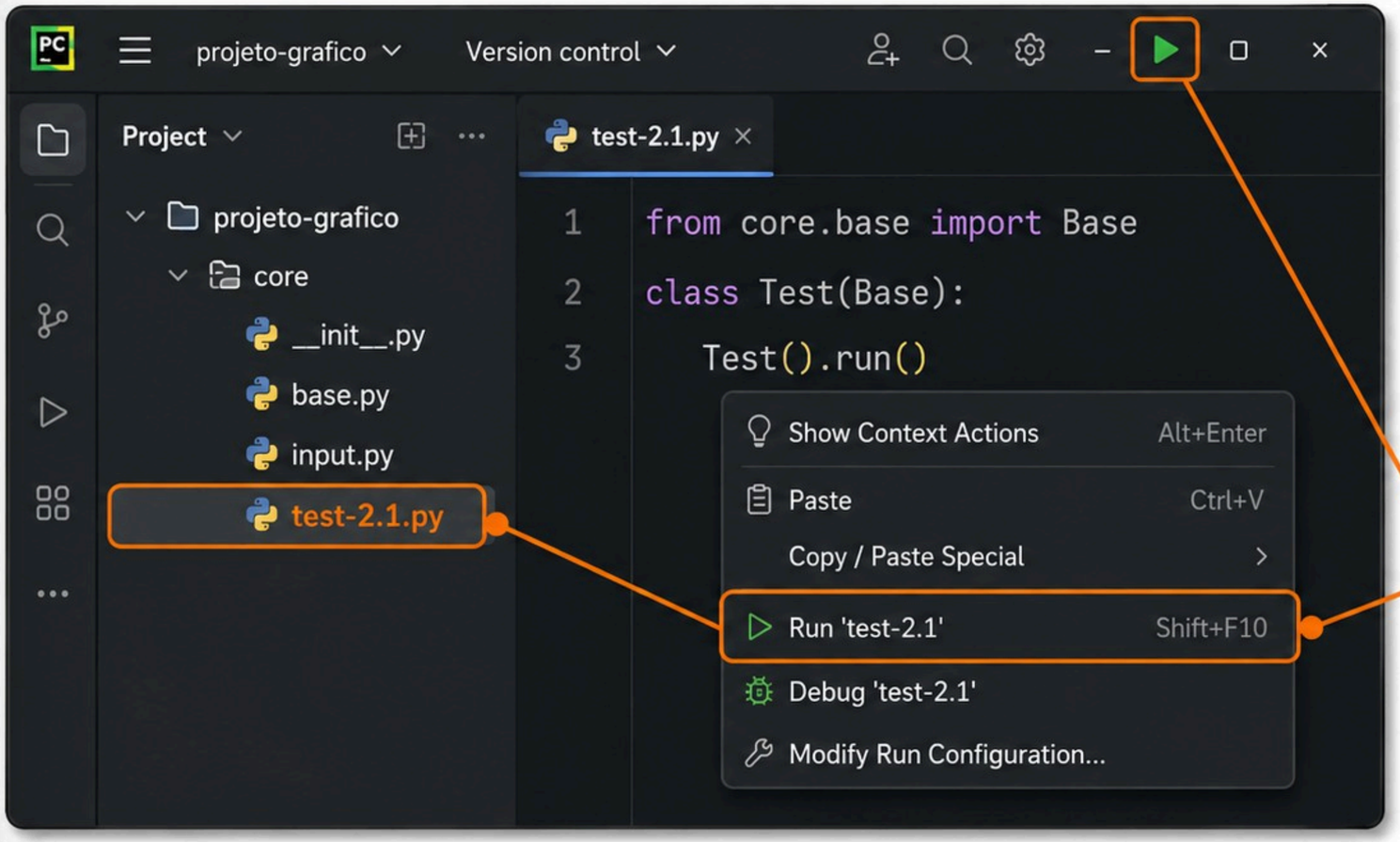
Input.quit = True

Base: running = False

pygame.quit()

Ideia-chave: Input não fecha a janela sozinho; ele comunica a decisão ao loop principal.

Executar o projeto



1. Na árvore de arquivos à esquerda, clique duas vezes em `test-2.1.py` para abri-lo.

2. Clique com o botão direito em qualquer lugar do código e escolha **Run 'test-2.1'**.

Alternativas: seta verde ▶ no canto superior direito ou ao lado de `Test().run()`.

DICA: Antes de executar, confirme que o arquivo aberto é `test-2.1.py`.

CONTATO

rodrigofernandes@unipac.br

OBRIGADO!!